Rapport du TP 01 — Hachage SHA-256, chaînage de blocs et minage

1. Introduction

Ce TP programme en Python les mécanismes de base d'une blockchain : hachage, chaînage de blocs, minage par preuve de travail et vérification d'intégrité. Il a été réalisé dans le module « Blockchain et confiance numérique » (Université Mundiapolis, filière Ingénierie Informatique, 5eme annee), encadré par M. Ayoub LOUJA.

Le fil conducteur est une question : comment empêcher la réécriture de l'historique d'un registre, sans autorité centrale pour le garantir ? Le TP y répond en deux temps. Le chaînage rend toute modification visible, et le minage la rend coûteuse.

Objectifs du TP

- calculer une empreinte SHA-256 avec la bibliothèque `hashlib` ;
- représenter un bloc par un dictionnaire et relier les blocs par leur empreinte ;
- programmer le minage par preuve de travail (Proof of Work) ;
- vérifier l'intégrité de la chaîne et mesurer le coût d'une falsification.

Registre des transactions utilisé dans tout le TP

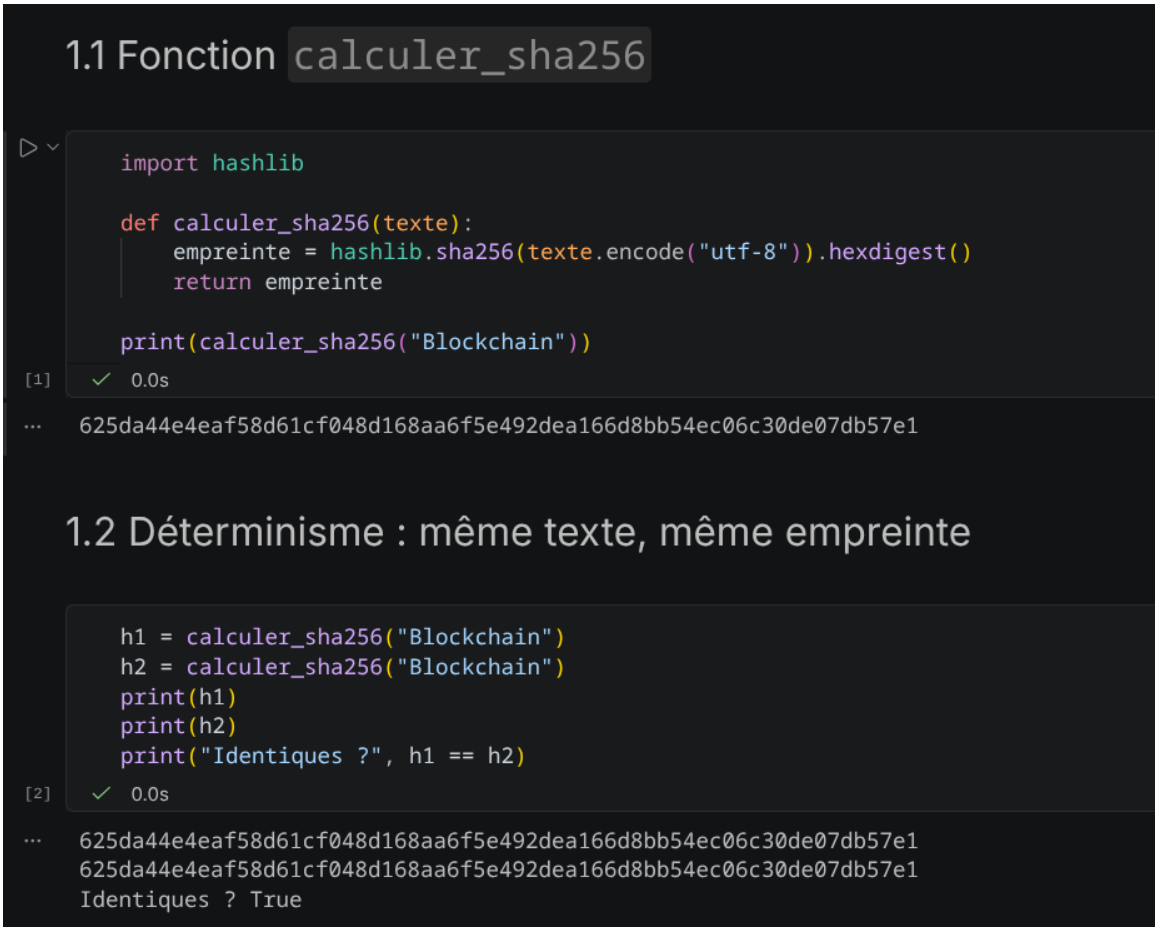
Bloc	Transaction
0	Bloc de genese Mundiapolis (aucune transaction)
1	Ali envoie 12$ a Sara
2	Sara envoie 5$ a Karim
3	Karim envoie 3$ a Nadia
4	Nadia envoie 7$ a Ali

Outils et méthode. Le code est écrit dans un notebook Jupyter, avec une cellule par question. Le notebook a été exécuté de haut en bas après un redémarrage du kernel (*Restart Kernel and Run All Cells*), pour que toutes les sorties viennent de la version finale du code. Le minage est déterministe : les nombres d'essais sont donc reproductibles, alors que les durées et le débit dépendent de la machine.

Le code source est disponible sur GitHub : https://github.com/meryem-el-fellah/TP01_blockchain/tree/main

2. Partie 1 — Hachage SHA-256

SHA-256 transforme un texte de taille quelconque en une empreinte de taille fixe : 256 bits, soit 64 caractères hexadécimaux. Trois propriétés de cette fonction servent de base à toute la blockchain, et le TP les vérifie une par une.



Implémentation (1.1). La fonction encode le texte en UTF-8 puis renvoie l'empreinte en hexadécimal. L'empreinte de "Blockchain" est `625da44e4eaf58d61cf048d168aa6f5e492dea166d8bb54ec06c30de07db57e1`.

1. Déterminisme (1.2). Le même texte donne toujours la même empreinte : deux calculs sur "Blockchain" sont identiques (`Identiques ? True`). Sans cela, personne ne pourrait recalculer un hash pour le vérifier.

2. Taille fixe (1.3). Un texte de 1, de 11 ou de 1000 caractères donne toujours une empreinte de 64 caractères.

1.3 Longueur de l'empreinte

```
for texte in ["a", "Mundiapolis", "x" * 1000]:
    print(len(texte), "caracteres ->", len(calculer_sha256(texte)))
```

[3] ✓ 0.0s

```
1 caracteres -> 64
11 caracteres -> 64
1000 caracteres -> 64
```

3. Effet avalanche (1.4 et 1.5). La moindre modification du texte change presque toute l'empreinte. La fonction `compter_differences` compare deux empreintes position par position :

Textes comparés	Positions différentes
"Bloc numero 1" et "Bloc numero 2"	59 sur 64
"Ali envoie 12$ a Sara" et "Ali envoie 13$ a Sara"	62 sur 64

Chaque caractère hexadécimal a une chance sur 16 d'être identique par hasard. Deux empreintes indépendantes ont donc environ 64 × 15/16 = 60 positions différentes : 59 et 62 sont très proches de cette valeur. Changer 12$ en 13$ ne laisse aucune ressemblance avec l'empreinte d'origine.

Conséquence pour la suite. L'effet avalanche a deux usages. Il rend toute falsification d'une transaction immédiatement visible dans le hash du bloc (partie 2). Il empêche aussi de deviner un hash à l'avance, ce qui oblige le minage à procéder par essais (partie 3).

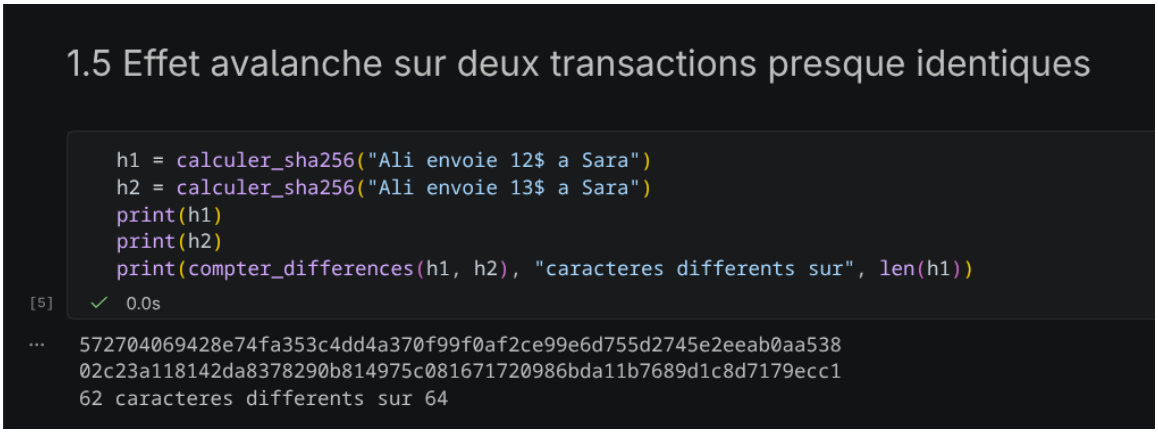
1.4 Effet avalanche : fonction `compter_differences`

```
def compter_differences(h1, h2):
    nb_differences = 0
    for i in range(len(h1)):
        if h1[i] != h2[i]:
            nb_differences += 1
    return nb_differences

h1 = calculer_sha256("Bloc numero 1")
h2 = calculer_sha256("Bloc numero 2")
print(h1)
print(h2)
print(compter_differences(h1, h2), "caracteres differents sur", len(h1))
```

[4] ✓ 0.0s

```
c17a909c42cb7d2acfb192bb9f75207a83cac136d2a006104a62b482b8b58793
587fa627c8742464522772db52e48634c1f40b5a4b7fc7e4a006c5f831059133
59 caracteres differents sur 64
```



3. Partie 2 — Représenter un bloc et chaîner

Un bloc est un dictionnaire à cinq clés, et la chaîne est une liste de blocs dans laquelle chaque bloc contient le hash du précédent. C'est ce lien qui crée le chaînage.

Clé	Contenu
`index`	position du bloc dans la chaîne (0, 1, 2, …)
`donnees`	la transaction enregistrée dans le bloc
`nonce`	compteur utilisé par le minage
`hash_precedent`	hash du bloc précédent (64 zéros pour le premier bloc)
`hash`	hash du bloc lui-même

Trois fonctions, trois rôles distincts (2.1 à 2.4).

- `creer_bloc` construit seulement le dictionnaire : `nonce` à 0 et `hash` vide. Elle ne calcule aucun hash.
- `calculer_hash_bloc` concatène, dans cet ordre, l'index, le nonce, les données et le hash précédent, puis renvoie le SHA-256 de ce texte. C'est **un seul calcul**, sans boucle.
- `ajouter_bloc` crée un bloc, calcule son hash et l'ajoute à la fin de la liste.



2.1 Fonction `creer_bloc`

```python
def creer_bloc(index, donnees, hash_precedent):
    bloc = {
        "index": index,
        "donnees": donnees,
        "nonce": 0,
        "hash_precedent": hash_precedent,
        "hash": ""
    }
    return bloc

print(creer_bloc(1, "Ali envoie 12$ a Sara", "abc"))
```

[7] ✓ 0.0s

```
{'index': 1, 'donnees': 'Ali envoie 12$ a Sara', 'nonce': 0, 'hash_precedent': 'abc', 'hash': ''}
```



2.2 Fonction `calculer_hash_bloc`

```python
def calculer_hash_bloc(bloc):
    contenu = str(bloc["index"]) + str(bloc["nonce"]) + bloc["donnees"] + bloc["hash_precedent"]
    return calculer_sha256(contenu)
```

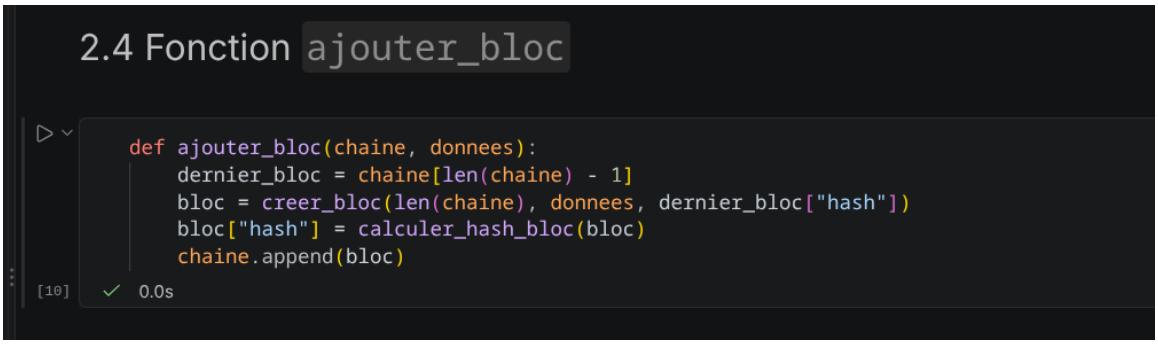
[8] ✓ 0.0s

2.3 Bloc de genèse

```python
genese = creer_bloc(0, "Bloc de genese Mundiapolis", "0" * 64)
genese["hash"] = calculer_hash_bloc(genese)
chaine = [genese]
print("Hash de la genese :", genese["hash"])
```

[9] ✓ 0.0s

```
Hash de la genese : fb75966eb8d3dda427fec6a298feed4a9d5b0973343ba8a697e710a04f101599
```



2.4 Fonction `ajouter_bloc`

```python
def ajouter_bloc(chaine, donnees):
    dernier_bloc = chaine[len(chaine) - 1]
    bloc = creer_bloc(len(chaine), donnees, dernier_bloc["hash"])
    bloc["hash"] = calculer_hash_bloc(bloc)
    chaine.append(bloc)
```

[10] ✓ 0.0s

Cette séparation est importante : calculer un hash (partie 2) et chercher un hash valide par essais répétés (minage, partie 3) sont deux opérations différentes. Un `calculer_hash_bloc` qui contiendrait la boucle de minage ne pourrait plus servir à vérifier une chaîne.

Dans `ajouter_bloc`, le `hash_precedent` est lu dans le bloc (`dernier_bloc["hash"]`) et non recalculé. La chaîne doit référencer le hash enregistré : un recalcul ferait un calcul inutile et masquerait un éventuel dernier bloc modifié. La fonction n'a pas de `return` : une liste passée en paramètre n'est pas copiée, donc `append` modifie directement la liste d'origine.

Bloc de genèse (2.3). Le premier bloc n'a pas de prédécesseur : son `hash_precedent` vaut 64 zéros. La ligne `chaine = [genese]` crée la liste qui contient déjà ce bloc. Elle est nécessaire, car `ajouter_bloc` a besoin d'un dernier bloc à qui rattacher le suivant. Le hash de la genèse est `fb75966eb8d3dda427fec6a298feed4a9d5b0973343ba8a697e710a04f101599`.

Construction de la chaîne (2.5). La boucle `for i in range(1, len(donnees_blocs))` démarre à 1, car la transaction d'index 0 a déjà servi à créer la genèse. L'affichage des 10 premiers caractères de chaque hash montre le chaînage :



Le `prec` de chaque bloc est le `hash` de la ligne précédente : modifier un bloc changerait son hash et romprait ce lien. Remarque d'exécution : relancer cette cellule sans refaire `chaine = [genese]` ajoute quatre blocs de plus à la même liste, ce qui explique une chaîne trop longue si la cellule est exécutée plusieurs fois.

Lecture des transactions (2.7 et 2.8). `lire_transaction` découpe le texte (`split(" ")`) et renvoie l'émetteur, le montant (entier) et le bénéficiaire. `calculer_soldes` part de 20 pour chacun et applique les quatre transactions, ce qui donne `{'Ali': 15, 'Sara': 27, 'Karim': 22, 'Nadia': 16}` : Ali a 20 − 12 + 7 = 15, et la somme des soldes reste 80.



Limite de cette partie. La chaîne est chaînée, mais aucun hash ne commence par des zéros : elle n'est pas "minée". Falsifier un bloc ne coûterait que le recalcul de quelques hashes, soit quelques microsecondes. La partie 3 ajoute la preuve de travail pour rendre cette modification coûteuse.

4. Partie 3 — Minage et preuve de travail

Miner un bloc, c'est essayer les nonces 0, 1, 2, 3… jusqu'à obtenir un hash qui commence par le nombre de zéros exigé (la difficulté). Le résultat est difficile à trouver, mais facile à vérifier : un seul calcul de hash suffit pour contrôler le nonce trouvé.

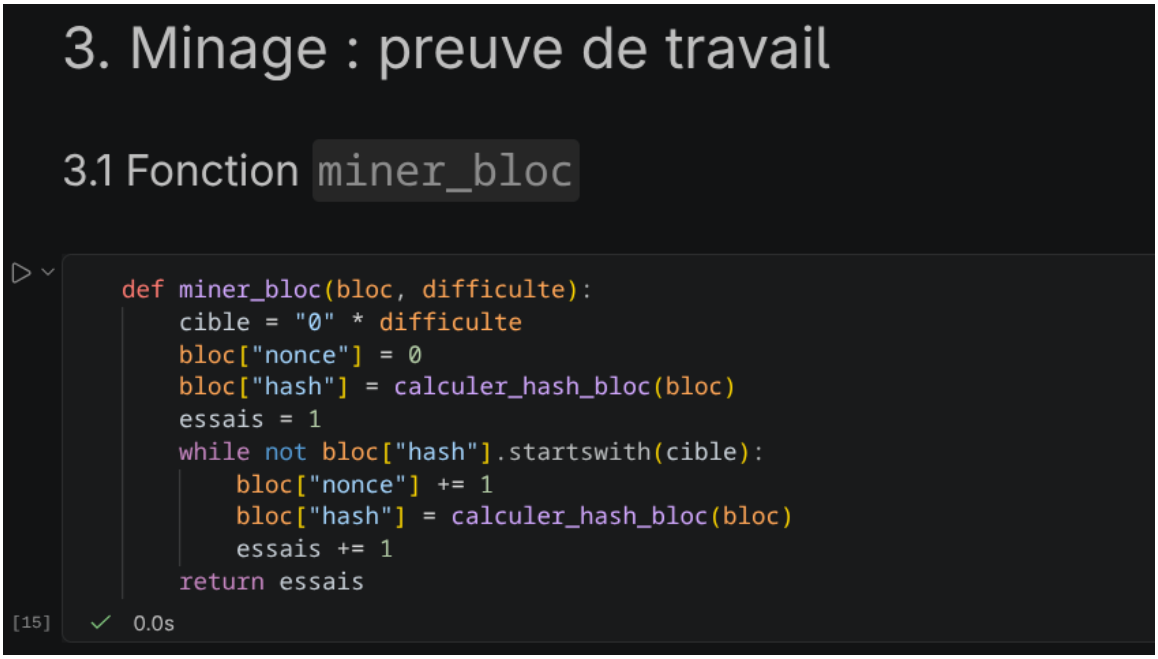
Le problème d'origine. Sans minage, ajouter ou réécrire un bloc ne coûte qu'un calcul de hash. Deux problèmes en découlent. D'abord, falsifier est gratuit : il suffit de recalculer les hashes des blocs suivants, ce qui est instantané. Ensuite, dans un réseau ouvert sans autorité centrale, rien ne désigne qui écrit le prochain bloc. La preuve de travail (Proof of Work) répond aux deux : ajouter un bloc doit exiger un travail réel.

Le principe. Grâce à l'effet avalanche, on ne peut pas prévoir quelle entrée donnera un hash commençant par `0000`. La seule méthode est d'essayer. Le **nonce** est un champ du bloc dont l'unique rôle est de faire varier le hash quand on le change. C'est un entier quelconque, sans limite (0, 1, 2, …, 41489, …), et non un chiffre de 0 à 9. Chaque caractère hexadécimal vaut 0 avec la probabilité 1/16, donc pour k zéros il faut en moyenne 16^k essais :

```
N_{moyen} = 16^{k} \qquad (k = 4 : 65\,536 \text{ essais} \;;\; k = 5 : 1\,048\,576 \text{ essais})
```

Miner est donc une recherche par force brute, et non la résolution d'un problème mathématique complexe. Ce n'est pas non plus du chiffrement. Dans une blockchain réelle, les pièces ne sont pas « extraites » : elles récompensent le mineur qui a produit un bloc valide que le réseau accepte.

La fonction `miner_bloc` (3.1). Elle remet le nonce à 0, calcule le hash, puis augmente le nonce et recalcule tant que le hash ne commence pas par la cible. Elle compte les essais (1 au départ) et les renvoie.



L'ordre est essentiel : le hash est **recalculé avant chaque test**. Tester d'abord un hash déjà stocké laisserait passer un bloc dont le contenu a changé sans qu'il soit re-miné, ce qui fausserait la partie 6.

Minage d'une genèse (3.2). Le TP demande un *nouveau* bloc de genèse, que l'on mine à la difficulté 4 : on trouve le nonce 41489, soit 41 490 essais, et le hash `0000eb2c58b542efffaf1dcdada2baf8832a9bb933949c41442410ec8a9e7889`. On ne réutilise pas la `genese` de la question 2.3, car `miner_bloc` modifie le bloc sur place et ce bloc est aussi `chaine[0]`. Le miner changerait son hash sans mettre à jour le `hash_precedent` du bloc 1, ce qui casserait la chaîne de la partie 2.

3.2 Miner un nouveau bloc de genèse (difficulté 4)

```
genese = creer_bloc(0, "Bloc de genese Mundiapolis", "0" * 64)
essais = miner_bloc(genese, 4)
print("Nonce trouve :", genese["nonce"])
print("Essais       :", essais)
print("Hash         :", genese["hash"])
```

16] ✓ 0.1s

```
Nonce trouve : 41489
Essais       : 41490
Hash         : 0000eb2c58b542efffaf1dcdada2baf8832a9bb933949c41442410ec8a9e7889
```

La fonction `construire_chaine` (3.3 et 3.4). Elle part d'une liste vide et d'un hash précédent égal à 64 zéros. Pour chaque transaction, elle crée le bloc, le mine, affiche une ligne `[mine]`, l'ajoute à la liste et retient son hash pour le bloc suivant.

3.3 Fonction `construire_chaine`

```
def construire_chaine(liste_donnees, difficulte):
    chaine = []
    hash_precedent = "0" * 64
    for i in range(len(liste_donnees)):
        bloc = creer_bloc(i, liste_donnees[i], hash_precedent)
        essais = miner_bloc(bloc, difficulte)
        print(f"[mine] Bloc {i} : {essais} essais -> {bloc['hash'][0:20]}...")
        chaine.append(bloc)
        hash_precedent = bloc["hash"]
    return chaine
```

[17] ✓ 0.0s

Bloc	Essais (difficulté 4)	Début du hash
0	41 490	`0000eb2c58b542efffaf`
1	48 145	`0000fb41e6cae14ae3b7`
2	22 968	`00008ddc86166eaece69`
3	13 919	`0000cc4bf72544ba469c`
4	20 150	`00009538d1e384c589b8`

Les lignes `[mine]` sont affichées par un `print` placé *dans* la fonction : elles réapparaissent donc à chaque appel de `construire_chaine`, y compris dans les parties 5 et 6.



Différence entre `ajouter_bloc` (2.4) et `construire_chaine` (3.3). Les deux construisent une chaîne en rattachant chaque bloc au précédent, mais la seconde ajoute la preuve de travail, ce qui change la nature de la chaîne.

	`ajouter_bloc` (2.4)	`construire_chaine` (3.3)
Minage	Non : hash calculé une fois, avec le nonce 0	Oui : `miner_bloc` cherche un nonce jusqu'à `0000…`
Point de départ	Une chaîne existante, qui contient déjà la genèse	Une liste vide : la genèse est traitée comme les autres blocs
Hash précédent	Lu dans la liste : `dernier_bloc["hash"]`	Gardé dans une variable, mise à jour à chaque tour
Résultat	Modifie la liste reçue, sans `return`	Crée et renvoie une nouvelle chaîne
Difficulté	Aucune	Paramètre `difficulte`, réutilisé aux parties 5 et 6 (4 puis 5)
Affichage	Aucun	Une ligne `[mine]` par bloc

La chaîne de la partie 2 n'est donc pas valide au sens de la preuve de travail : le hash de sa genèse, `fb75966e…`, ne commence pas par `0000`, et `verifier_chaine(chaine, 4)` s'arrêterait dès le bloc 0. La chaîne de 3.4 passe les contrôles. Les blocs doivent aussi être minés dans l'ordre, puisque le bloc `i` contient le hash *miné* du bloc `i-1`. En résumé, `construire_chaine` généralise `ajouter_bloc` en y ajoutant le minage. On aurait pu fusionner les deux avec un paramètre de difficulté ; le TP les sépare pour étudier d'abord le chaînage, puis le minage.

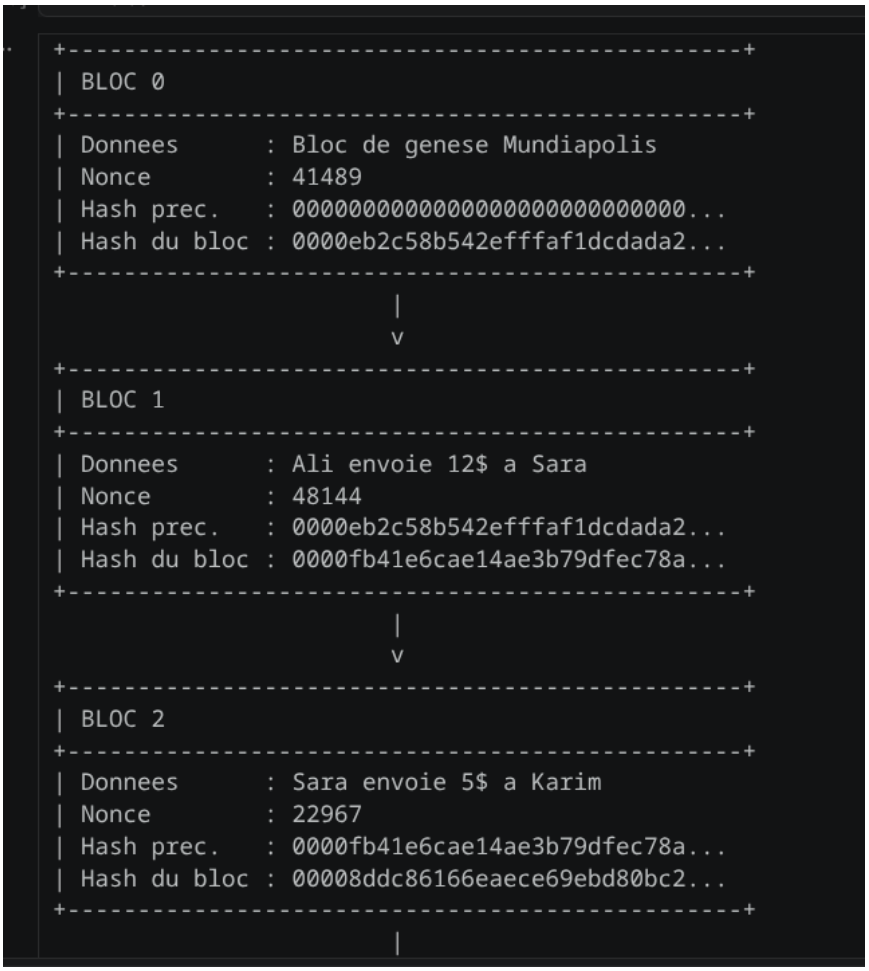
5. Partie 4 — Visualisation et vérification de la chaîne

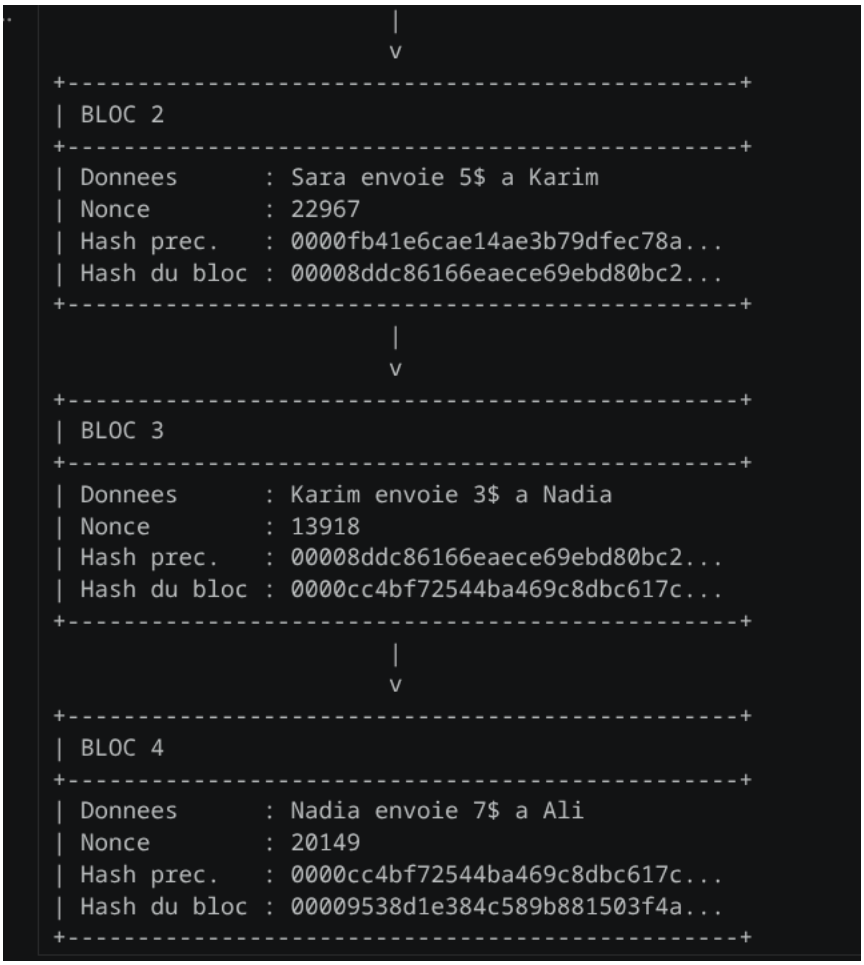
La fonction `verifier_chaine` applique trois contrôles à chaque bloc, et la chaîne n'est valide que si tous réussissent. Au premier contrôle qui échoue, la fonction affiche un message et renvoie `False`.

Affichage (4.1). `afficher_chaine` dessine chaque bloc dans un cadre de tirets, avec les hashes tronqués à 28 caractères (`[0:28]`), et place une flèche entre deux blocs, sauf après le dernier (`if i < len(chaine) - 1`).



La largeur du cadre (48) vient d'un calcul : la ligne la plus longue, `"| Hash du bloc : "` (17 caractères) + 28 caractères de hash + `"..."`, fait 48 caractères et tient dans le cadre. Les sorties semble comme suivant:


+---+
| BLOC 0
+---+
| Donnees   : Bloc de genese Mundiapolis
| Nonce   : 41489
| Hash prec.   : 00000000000000000000000000...
| Hash du bloc : 0000eb2c58b542efffaf1dcdada2...
+---+
|
v
+---+
| BLOC 1
+---+
| Donnees   : Ali envoie 12$ a Sara
| Nonce   : 48144
| Hash prec.   : 0000eb2c58b542efffaf1dcdada2...
| Hash du bloc : 0000fb41e6cae14ae3b79dfec78a...
+---+
|
v
+---+
| BLOC 2
+---+
| Donnees   : Sara envoie 5$ a Karim
| Nonce   : 22967
| Hash prec.   : 0000fb41e6cae14ae3b79dfec78a...
| Hash du bloc : 00008ddc86166eaece69ebd80bc2...
+---+
|



Vérification (4.2). Les trois contrôles sont exécutés dans cet ordre :

Contrôle	Ce qu'il garantit	Message en cas d'échec
`calculer_hash_bloc(bloc)` égal à `bloc["hash"]`	Le contenu n'a pas été modifié	Bloc i : le hash recalcule ne correspond pas au hash stocke
`bloc["hash"]` commence par la cible	Le travail de minage a été fait	Bloc i : la preuve de travail n'est pas satisfaite
Pour i > 0, `hash_precedent` égal à `chaine[i - 1]["hash"]`	Le bloc est bien rattaché à son prédécesseur	Bloc i : hash_precedent different du hash du bloc i-1

```
def verifier_chaine(chaine, difficulte):
    cible = "0" * difficulte
    for i in range(len(chaine)):
        bloc = chaine[i]
        if calculer_hash_bloc(bloc) != bloc["hash"]:
            print(f"Bloc {i} : le hash recalcule ne correspond pas au hash stocke")
            return False
        if not bloc["hash"].startswith(cible):
            print(f"Bloc {i} : la preuve de travail n'est pas satisfaite")
```

```
            return False
        if i > 0 and bloc["hash_precedent"] != chaine[i - 1]["hash"]:
            print(f"Bloc {i} : hash_precedent different du hash du bloc {i - 1}")
            return False
    return True
```

Sur la chaîne minée à la difficulté 4, `verifier_chaine(chaine, 4)` affiche `Chaine valide ? True` (4.3).

[image: 15.png]

Produire un bloc et valider un bloc. Ces deux rôles sont distincts. `miner_bloc` *produit* un bloc valide : c'est le travail du mineur, qui demande des milliers d'essais. `verifier_chaine` *valide* : c'est le travail de n'importe quel nœud du réseau, qui recalcule un hash par bloc et compare. Cette dissymétrie, « difficile à trouver, facile à vérifier », est ce qui permet à un réseau d'accepter un bloc sans faire confiance à celui qui l'a miné.

Limite du TP. Dans un réseau réel, la validation contrôle aussi les transactions : l'émetteur a-t-il un solde suffisant, la transaction est-elle signée. Ce contrôle n'existe pas ici. Une transaction comme « Ali envoie 500$ a Sara », alors qu'Ali n'a que 20$, ne serait pas rejetée : `calculer_soldes` afficherait simplement un solde négatif.

6. Partie 5 — Influence de la difficulté

Ajouter un zéro à la cible multiplie le travail par 16 environ : le total passe de 146 672 essais (cible `0000`) à 2 228 145 essais (cible `00000`), soit un rapport de 15,2.

Compter les essais (5.1). `construire_chaine` affiche le nombre d'essais de chaque bloc mais ne le conserve pas : la variable `essais` disparaît à la fin de chaque tour, et le bloc n'a que cinq clés. Une fois la chaîne construite, l'information n'existe plus que sous forme de texte à l'écran. Elle se retrouve pourtant dans le nonce : le minage part de 0 et augmente de 1 à chaque essai, donc un bloc dont le nonce final vaut `n` a demandé exactement `n + 1` essais (on compte aussi l'essai du nonce 0).

[image: 16.png]

Sur la chaîne de difficulté 4, les nonces 41489, 48144, 22967, 13918 et 20149 donnent 41 490 + 48 145 + 22 968 + 13 919 + 20 150 = 146 672 essais. La formule `n + 1` n'est exacte que parce que `miner_bloc` remet le nonce à 0 au départ. Cette fonction marche sur n'importe quelle chaîne déjà construite, y compris après un re-minage.

[image: 17.png]

Mesures (5.2 à 5.3). La chaîne est construite aux difficultés 4 puis 5, avec `time.time()` avant et après l'appel pour mesurer la durée. Les lignes `[mine]` réapparaissent, puisque le `print` est dans `construire_chaine`.

[image: 18.png]

Tableau récapitulatif (5.5).

Bloc (transaction)	Essais, cible 0000	Essais, cible 00000	Rapport
0 : genèse	41 490	985 225	23,7
1 : Ali → Sara 12$	48 145	272 033	5,7
2 : Sara → Karim 5$	22 968	169 506	7,4
3 : Karim → Nadia 3$	13 919	446 240	32,1
4 : Nadia → Ali 7$	20 150	355 141	17,6
Total	**146 672**	**2 228 145**	**15,2**
Durée (ma machine)	0,18 s	1,63 s	9,1

Lecture des résultats.

- Bloc par bloc, le rapport varie beaucoup (de 5,7 à 32,1), car le minage est un tirage aléatoire : un bloc peut avoir de la chance ou non. Sur le total, la loi des grands nombres rapproche le rapport de la valeur théorique 16 : on obtient 15,2.
- Le rapport des durées (9,1) s'écarte davantage de 16, parce que la durée est une mesure moins fiable : à la difficulté 4, le minage ne dure qu'une fraction de seconde. Le nombre d'essais, lui, est reproductible, et c'est lui qui sert de référence.
- Le débit de la machine est le nombre d'essais divisé par la durée (`total5 / duree5`), soit environ 1,4 million de hash par seconde ici. Une cible de 8 zéros demanderait en moyenne 16^8 ≈ 4,3 milliards d'essais, soit environ 3 100 secondes (moins d'une heure) sur cette machine. Chaque zéro de plus fait ainsi exploser le temps nécessaire.

[image: 19.png]

Dans la réalité. Le TP fixe la difficulté à la main (4 puis 5). Dans Bitcoin, c'est le réseau qui règle ce curseur : tous les 2016 blocs, il compare le temps réel au temps visé et ajuste la difficulté pour qu'un bloc soit trouvé en 10 minutes environ. Le nombre 2016 vient d'un calcul (14 jours × 24 heures × 6 blocs par heure), c'est un choix de conception et non une nécessité mathématique. Un intervalle plus court réagirait au simple hasard des tirages, un intervalle plus long réagirait trop lentement aux changements de puissance de calcul.

7. Partie 6 — Falsification d'un montant

Modifier un montant casse la chaîne, et la réparer oblige à re-miner le bloc modifié et tous les blocs suivants : plus la transaction est ancienne, plus la falsification coûte cher. Le scénario est le suivant : Ali modifie le bloc 1 de "Ali envoie 12$ a Sara" en "Ali envoie 2500$ a Sara", sans toucher au reste de la chaîne.

Falsification sans rien recalculer (6.1 et 6.2). Avant la modification, `verifier_chaine(chaine, 4)` renvoie `True`. Après, elle renvoie `False` avec le message « Bloc 1 : le hash recalcule ne correspond pas au hash stocke ». Les deux hashes du bloc 1 montrent l'effet avalanche : le hash stocké

`0000fb41e6cae14ae3b79dfec78a52e6504a09d7e3cbe8aba28b9263eaa215a9` n'a plus rien à voir avec le hash recalculé `8e265235c949f39c7809bf18bc9df3d44e8fe463def4471654281f38e1e57e6d`, et celui-ci a perdu ses zéros de tête.

[image: 20.png]

Re-miner seulement le bloc 1 (6.3). Le re-minage demande 31 895 essais et rend le bloc 1 cohérent avec ses nouvelles données (hash `0000f5c9f0d67d186e41…`). Mais son hash a changé, alors que le bloc 2 contient toujours l'ancien dans son `hash_precedent`. La vérification échoue donc un cran plus loin : « Bloc 2 : hash_precedent different du hash du bloc 1 ». C'est l'**effet domino** : le lien du bloc suivant est rompu, puis celui du suivant, jusqu'au dernier bloc.

[image: 21.png]

Re-miner toute la fin de la chaîne (6.4 et 6.5). La fonction `reminer_a_partir_de` parcourt les blocs à partir de `debut`. Pour chacun, elle recopie d'abord le hash du bloc précédent dans son `hash_precedent`, puis elle le mine, et elle additionne les essais.

[image: 22.png]

Bloc re-miné	Essais	Nouveau hash (début)
1	31 895	`0000f5c9f0d67d186e41`
2	32 567	`00003952799ef40f2394`
3	5 397	`00004ff8de4bc444ef57`
4	78 354	`000042c383bce62f9ca8`
Total	**148 213**	

Après ce re-minage complet, `verifier_chaine` renvoie de nouveau `True`. Les nombres d'essais des blocs 2 à 4 sont différents de ceux de la chaîne d'origine : le `hash_precedent` fait partie du contenu d'un bloc, donc quand il change, la recherche du nonce repart de zéro. **Le travail déjà fait n'est pas réutilisable.** L'ordre des instructions compte : si le `hash_precedent` n'est pas mis à jour *avant* le minage, le bloc garde le même contenu, le minage retrouve le même nonce et le même hash qu'avant, et la chaîne reste invalide. Retrouver exactement les mêmes essais est le signe que rien n'a changé.

[image: 23.png]

Falsifier le dernier bloc (6.6). Si Ali falsifie le bloc 4 ("Nadia envoie 700$ a Ali"), aucun bloc ne dépend de lui : il suffit de le re-miner, soit 29 829 essais.

[image: 24.png]

Une difficulté de plus (6.7). À la difficulté 5, falsifier le bloc 1 demande de re-miner les blocs 1 à 4 : 855 060 + 1 202 634 + 176 401 + 198 623 = 2 432 718 essais, soit un facteur de 16,4 par rapport à la difficulté 4.

Falsification	Blocs à re-miner	Difficulté	Essais
Bloc 1 (le plus ancien)	1, 2, 3 et 4	4	148 213
Bloc 4 (le dernier)	4	4	29 829
Bloc 1	1, 2, 3 et 4	5	2 432 718

[image: 25.png]

Ce qu'il faut en retenir.

- **Le coût dépend du nombre de blocs qui suivent.** Falsifier le bloc 1 oblige à refaire quatre blocs, presque autant de travail que pour construire la chaîne d'origine (146 672 essais). Falsifier le dernier n'en demande qu'un. Le rapport est d'environ 5 et non exactement 4, car le minage est aléatoire.
- **Plus une transaction est ancienne, mieux elle est protégée.** Chaque bloc ajouté après elle augmente le travail nécessaire pour la falsifier. C'est pourquoi, dans Bitcoin, on attend en général 6 blocs de confirmation avant de considérer une transaction comme définitive.
- **La blockchain n'est pas impossible à modifier, elle est difficile à modifier.** Une falsification reste réalisable, mais son coût croît avec le nombre de blocs et avec la difficulté (facteur 16 par zéro). Dans un réseau réel, le reste du réseau continue en plus d'ajouter des blocs pendant que l'attaquant re-mine.

8. Conclusion

Ce TP montre qu'une blockchain n'est pas impossible à modifier, mais que sa modification est rendue coûteuse par la combinaison de trois mécanismes. Le hachage SHA-256 rend toute modification visible grâce à l'effet avalanche. Le chaînage propage cette modification à tous les blocs suivants, puisque chaque bloc contient le hash du précédent. La preuve de travail transforme cette propagation en un coût mesurable : 148 213 essais pour falsifier le bloc 1 contre 29 829 pour le dernier, et un facteur 16 environ pour chaque zéro de difficulté ajouté.

Concepts à retenir.

- Miner, c'est chercher par essais un nonce qui donne un hash commençant par des zéros : difficile à trouver, facile à vérifier.
- `calculer_hash_bloc` fait un seul calcul, `miner_bloc` répète ce calcul jusqu'au succès, et `verifier_chaine` rejoue des contrôles sans rien modifier.
- `construire_chaine` généralise `ajouter_bloc` en y ajoutant le minage.
- Le travail de minage d'un bloc n'est pas réutilisable dès que son contenu, y compris le hash précédent, change.

Limites du TP. La difficulté est fixée à la main et il n'y a qu'une seule machine, sans réseau ni consensus entre plusieurs nœuds. Les transactions ne sont ni signées ni contrôlées (aucun test de solde suffisant).

Enfin, les durées mesurées dépendent de la machine, alors que les nombres d'essais sont reproductibles.

Remarques de méthode. Deux points ont demandé de l'attention en Jupyter. Une fonction utilisée par les cellules suivantes est celle de la *dernière cellule `def` exécutée* : après une modification, il faut la ré-exécuter, puis relancer les cellules qui l'appellent. Et une liste, comme `chaine`, garde son contenu entre deux exécutions : relancer une cellule qui y ajoute des blocs la rallonge. Pour obtenir des sorties cohérentes, le notebook final a été exécuté après un redémarrage complet du kernel.

Code source. Le notebook et l'énoncé rempli sont disponibles sur GitHub : https://github.com/meryem-el-fellah/TP01_blockchain/blob/main/TP01_Blockchain.ipynb